

Technical and Strategic Implementation Blueprint: "Which partner is at fault?" (AI Relationship Toxicity Analyzer)

The intersection of digital forensics, natural language processing, and behavioral psychology has yielded a highly complex frontier for consumer applications. The mobile application "Which partner is at fault?" (Subtitle: AI relationship toxicity analyzer) represents a sophisticated analytical platform designed to ingest raw communication data across multiple disparate channels, process these linguistic artifacts through large language models (LLMs) against specific psychological metrics, and output highly structured, evidentiary PDF reports. Deploying this application globally requires navigating severe operating system privacy sandboxes, identifying hyper-efficient AI inference APIs, and establishing a rigorous financial and corporate infrastructure to manage the dual-tier subscription economics. The application relies on a \$9.99 per month foundational tier for the analysis of native text messages (SMS/iMessage) and a \$19.99 per month premium tier capable of ingesting email threads and third-party chat environments (WhatsApp, Instagram, LinkedIn, Telegram, Messenger). To facilitate the immense computational overhead of API token expenditures alongside continuous subscription revenue, a localized corporate and financial hub must be established in Lady Lake, Florida. This exhaustive report details the architectural, programmatic, psychological, and fiscal methodologies required to execute this deployment in 2026.

Cross-Platform Framework Selection and UI Topography

The architectural foundation of the application dictates its rendering performance, its capacity to interface with native device file systems, and its programmatic document generation capabilities. The mandate requires deploying to both Android and iOS while maintaining a specific left/right pane graphical user interface (GUI). The evaluated frameworks include B4X, React Native, and Flutter.

The B4X ecosystem, utilizing B4A for Android and B4i for iOS, provides a rapid application development environment based on a syntax similar to Visual Basic. Recent updates within the B4X community have introduced specific libraries for OpenAI API interactions and root-level USB logging. However, the B4X framework fundamentally lacks the deep declarative UI paradigms required to seamlessly execute dynamic split-pane interfaces across varying screen resolutions. Furthermore, its ecosystem for complex programmatic PDF generation with embedded images and active hyperlinks is highly nascent compared to modern alternatives, rendering it unsuitable for an application heavily reliant on advanced document synthesis. React Native operates by communicating with native OEM widgets via a JavaScript bridge. While architectural overhauls in 2026 have mitigated historical latency, React Native remains fundamentally dependent on the host operating system for visual rendering. This approach excels for content-driven applications managed by teams deeply entrenched in JavaScript, but it introduces significant friction when handling massive, media-heavy chat threads that require granular memory management. Furthermore, generating PDFs in React Native frequently necessitates bridging to native iOS CoreGraphics or Android PdfDocument APIs, fracturing the codebase and complicating the embedding of inline chat images.

Flutter represents the optimal architectural choice for this deployment. Unlike React Native, Flutter bypasses OEM widgets entirely, rendering every pixel directly to the screen utilizing the Skia graphics engine—and increasingly, the Impeller rendering engine—which guarantees absolute visual determinism across all operating systems. This rendering architecture is explicitly designed for the requested left/right pane UI topography. By utilizing layout widgets such as Row, Expanded, and LayoutBuilder, the application can seamlessly maintain a persistent master-detail split-view on tablets and desktop deployments, while elegantly collapsing into a swipeable tab view on smaller mobile screens. Crucially, Flutter maintains an absolute monopoly on cross-platform PDF generation via the pdf and printing packages. These libraries permit the construction of robust, multi-page PDF documents utilizing a widget tree syntax (pw.Widget) that directly mirrors the application's primary UI code. This ecosystem inherently supports Unicode, right-to-left text, dynamic tabular formatting, hyperlinking, and the embedding of binary image data (JPEGs and PNGs) via MemoryImage processing. The capacity to stream a massive array of parsed chat messages directly into a formatted PDF canvas without relying on native bridging makes Flutter the definitive foundational framework.

Framework Evaluation Summary

Framework	Rendering Pipeline	PDF Generation Capability	Cross-Platform Ecosystem UI	Ideal Use Case
B4X	Native wrapper via Basic syntax	Limited; requires extensive custom libraries	Functional but dated visual paradigms	Hobbyist applications; basic utility apps.
React Native	JS Bridge to native OEM widgets	Moderate; often relies on fragile native bridges	High native feel; varies by OS platform	Apps requiring deep web-infrastructure integration.
Flutter	Direct pixel rendering (Skia/Impeller)	Exceptional; native pw.Widget tree construction	Perfect determinism; ideal for Split-Pane layouts	High-performance apps requiring custom branding and local document synthesis.

Forensic Data Extraction Modalities

The application's monetization strategy is strictly bifurcated into two tiers. The foundational \$9.99 per month subscription provides the capability to ingest and analyze native device text messages. The premium \$19.99 per month subscription expands this ingest capacity to include enterprise email (Gmail, Outlook, IMAP) and encrypted third-party messaging ecosystems. The engineering pathways to access these disparate data silos are fundamentally different.

Tier 1 (\$9.99/mo): Native Text Message Forensics

Extracting native SMS and iMessage data directly from the host device presents severe, constantly evolving regulatory and operating system challenges. Within the Android ecosystem, historical applications enjoyed broad access to communication logs via the READ_SMS permission. However, Google Play policies implemented and refined through 2025 and 2026 have severely restricted these vectors to combat financial spyware and

malicious actors. Currently, an Android application may only utilize READ_SMS or access call logs if it has been explicitly selected by the user as the system's default handler for text messages. Therefore, to remain compliant with Google Play distribution rules, the application must temporarily prompt the user to designate it as the default SMS application, execute the SQLite extraction of the device's SMS inbox, and subsequently prompt the user to return default status to their original messaging application. Applications failing to submit a rigorous Permissions Declaration Form verifying this "critical core" use case face immediate removal. Additionally, developers must account for recent OS-level privacy enhancements, such as the deprecation of the Android ID as a persistent identifier, requiring the use of alternative cryptographic hashes to anonymously link extracted text threads to specific users.

The iOS environment presents an even more impenetrable sandbox. Apple strictly prevents third-party applications from programmatically reading the native Messages database (sms.db). While the European Union's Digital Markets Act (DMA) has forced Apple to introduce over 600 new interoperability APIs in iOS 18 and iOS 19, these APIs are strictly geofenced to the EU and primarily focus on alternative app distribution, default browser selection, and accessory interoperability (such as Bluetooth headphone switching and third-party smartwatch notification displays). They do not grant global, unrestricted access to native iMessage histories. Consequently, providing native text analysis for iOS users necessitates a secondary extraction vector. The architectural workaround requires users to interface with a desktop environment. Utilizing software such as iMazing, iBackup Extractor, or open-source Python iMessage Exporters, the user must generate an unencrypted local iTunes or Finder backup of their iPhone. The desktop software parses the underlying sms.db and manifest.db SQLite files to expose the raw conversation threads. The mobile application will provide an interface allowing users to upload the resulting CSV, JSON, or TXT exports directly into the Flutter application for ingestion, mapping the external data into the app's internal chat models. While this introduces a friction point in the user experience, the legal and evidentiary value of the resulting analysis justifies the procedural complexity.

Tier 2 (\$19.99/mo): Email and Third-Party Interoperability

Analyzing third-party ecosystems (WhatsApp, Instagram, Telegram, LinkedIn, Messenger) and email threads (Gmail, Outlook) requires bypassing closed, proprietary networks. Implementing independent OAuth flows and reverse-engineering private APIs for six distinct communication platforms is architecturally unsustainable. The solution relies on integrating a unified communication API aggregator. By 2026, Unipile serves as the premier enterprise solution for this exact methodology.

Unipile provides a centralized REST API architecture that collapses over 500 provider-specific endpoints into approximately 150 unified endpoints. The integration allows the Flutter application to offer a white-labeled "Hosted Auth" flow, enabling users to scan QR codes or input credentials to link their external accounts directly within the app's left pane UI without navigating away from the application.

For messaging platforms, calling the unified GET /api/v1/chats and GET /api/v1/messages endpoints allows the application to retrieve comprehensive historical conversation threads, including metadata, sender profiles, and binary attachments. Furthermore, Unipile supports real-time webhooks (message_received, message_edited, message_deleted), which are pushed to the application backend in under 500 milliseconds, allowing the toxicity analyzer to provide live, dynamic scoring of ongoing hostile conversations.

Crucially, the \$19.99 tier promises the ingestion of email threads. Unipile's Email API fundamentally expands the app's capability by offering unified support for Gmail, Microsoft Outlook, and custom IMAP servers. Instead of mapping three disparate schemas, the application utilizes a single normalized data model to ingest long-form email correspondence.

For Gmail and Outlook, the integration relies on secure OAuth 2.0 flows ensuring no passwords are stored, while supporting incremental delta synchronization and label/folder management to isolate specific conversations. This is highly critical for users facing workplace hostility or legal disputes occurring over corporate email, bridging the gap between personal relationship toxicity and professional harassment.

The economic viability of relying on Unipile is highly favorable. Unipile utilizes a per-linked-account billing structure. The base platform fee is €49.00 per month, which includes the first 10 linked accounts. Beyond this threshold, each additional linked account (e.g., one WhatsApp number or one Gmail inbox) costs between €3.00 and €5.00 per month, heavily discounting at larger volumes. Assuming the average \$19.99/mo premium subscriber connects two accounts (e.g., WhatsApp and Gmail), the API overhead remains strictly under €10.00, preserving a massive gross margin for LLM inference and profitability.

Unipile Email API Capabilities

Email Provider	Authentication Method	Synchronization Features	Webhook Support	Primary Use Case within App
Gmail	OAuth 2.0 (No password storage)	Incremental delta sync, Labels	Real-time push (New Email)	Personal correspondence, long-form relationship arguments.
Outlook / M365	Microsoft Graph OAuth	Folder & Category sync	Real-time push	Corporate communication, evidence gathering.
Custom IMAP	App Password / Credentials	Full mailbox sync, Folders	Polling / Events	Legacy email providers, specific domain hosting.

Algorithmic Token Economics and LLM Optimization

The core computational engine of the application relies on passing massive volumes of ingested text—frequently amounting to tens of thousands of words per relationship timeline—into a Large Language Model (LLM) for psychological evaluation. Generating a complex, multi-metric PDF report requires an API that balances high reasoning capabilities with extreme token efficiency. By April 2026, the economics of LLM inference have collapsed dramatically, enabling startups to deploy capabilities that required enterprise budgets just two years prior.

The industry relies on a bifurcated pricing model, charging independently for input tokens (the raw text thread and the system prompt provided to the model) and output tokens (the analytical report generated by the model). Because this application ingests massive contexts but outputs relatively brief, structured reports, optimizing for input token pricing is the primary economic imperative.

Premium tier models, such as OpenAI's GPT-5.4 Pro and Anthropic's Claude Opus 4.6, command exorbitant prices, ranging from \$15.00 to \$30.00 per million input tokens. While these models offer unparalleled performance in high-stakes legal reasoning, their cost profile instantly destroys the profit margins of a \$9.99 per month subscription model if a user decides to analyze a multi-year chat history. Conversely, Google's Gemini 2.5 Flash-Lite establishes the absolute price floor at an astonishing \$0.10 per million input tokens, paired with a massive 1-million token context window. While highly cost-effective, hyper-fast models like Flash-Lite occasionally

struggle with the deep, abstract reasoning required to detect subtle psychological abuse, such as distinguishing chronic sarcasm from gaslighting.

The optimal architectural decision is the implementation of DeepSeek V3.2. As of early 2026, DeepSeek disrupted the API pricing floor, offering flagship-level reasoning capabilities for \$0.14 to \$0.28 per million input tokens, and \$0.38 to \$0.42 per million output tokens. DeepSeek V3.2 provides a 128K context window—sufficient to hold approximately 90,000 words of chat history in a single API call. Furthermore, DeepSeek’s underlying architecture has proven highly capable of traversing meta-toxic knowledge graphs, allowing it to accurately extract subtle toxic entities and implicit semantic threats that naive retrieval-augmented generation systems miss.

By utilizing DeepSeek V3.2, a user analyzing an exhaustive 50,000-word SMS thread (approximately 65,000 tokens) incurs an input cost of roughly \$0.018. If the application prompts the model to generate a structured 1,000-word diagnostic output, the output cost equates to roughly \$0.005. The total inference cost per analysis sits below three cents. This staggering efficiency ensures the \$9.99 per month foundational subscription remains overwhelmingly profitable, regardless of user engagement frequency.

LLM Token Pricing Matrix (April 2026)

Model Architecture	Input Cost (per 1M tokens)	Output Cost (per 1M tokens)	Context Window	Application Viability
Gemini 2.5 Flash-Lite	\$0.10	\$0.30	1.0M	High for basic data extraction, poor for deep psychological reasoning.
DeepSeek V3.2	\$0.14 - \$0.28	\$0.38 - \$0.42	128K	Optimal choice; massive savings with high-tier cognitive routing.
GPT-5.2 Instant	\$1.50	\$4.50	256K	Mediocre value proposition compared to emerging models.
Claude Sonnet 4.6	\$3.00	\$15.00	200K	Excellent nuance, but too expensive for a \$9.99/mo margin.
GPT-5.4 Pro	\$30.00	\$80.00	200K	Cost prohibitive; instantly bankrupts the subscription model.

The Psychological Forensic Framework: 50 Analytical Metrics

A core feature of the application allows users to select 10 out of 50 highly specific psychological and linguistic metrics for the LLM to process against the ingested text log. Human analysis of digital communication frequently overlooks cumulative abuse, as toxic patterns are often dispersed across months of mundane interaction. AI analysis, utilizing deep semantic

embeddings and emotion layers, excels at identifying these invisible tactics—the psychological patterns that shift blame, reframe harm, and erode self-worth. The 50 metrics are categorized into five distinct diagnostic domains to structure the LLM's system prompt effectively. The AI will output a normalized score (0.0 to 1.0) for each selected metric, flagging specific quotations as forensic evidence for the PDF report.

Domain 1: Coercive Control and Reality Distortion

#	Metric	Clinical Definition & AI Detection Mechanism
1	Gaslighting	Denying a partner's reality or memories. The AI detects semantic phrases like "You're imagining things" or "You remember it wrong" paired with high emotional distress from the recipient.
2	DARVO	<i>Deny, Attack, Reverse Victim and Offender.</i> The AI analyzes sequential logic where the initiator of a complaint is suddenly forced to defend themselves against a counter-accusation.
3	Love Bombing	An intense early rush of overwhelming affection that abruptly ceases. The AI tracks positive sentiment volume clustering in the first 30 days followed by steep, permanent algorithmic decline.
4	Triangulation	Introducing a third party into text arguments. The AI flags recurring named entity recognition (NER) of outside individuals used to incite jealousy or diminish the partner's worth.
5	Isolating Language	Systematic attempts to sever the partner's external support systems. The AI identifies hostile sentiment directed at the partner's friends or family (e.g., "I hate your friends").
6	Guilt Tripping	Leveraging emotional obligation. The LLM detects conditional logic tied to affection, such as "If you loved me, you would do what I want".
7	Ultimatums	Framing minor conflicts as

#	Metric	Clinical Definition & AI Detection Mechanism
		relationship-ending scenarios. The AI isolates conditional threats related to relationship termination to force immediate compliance.
8	Goalpost Shifting	Continuously changing the criteria for satisfaction. The AI detects sequential demands where previous compliance is immediately met with a new, harder requirement.
9	Resource Manipulation	Financial or logistical coercion. The AI flags vocabulary surrounding money, tracking, or access control utilized in a demanding or restrictive context.
10	Recovery-Phase Manipulation	Utilizing gentle, affectionate language immediately following an abusive episode to reset the cycle of abuse and prevent the victim from leaving.

The detection of coercive control requires the LLM to analyze the broader contextual timeline rather than isolated sentences. For instance, DARVO is a sequence, not a keyword. The LLM must be prompted to recognize structural shifts in the argument flow where the burden of proof is maliciously transferred. Similarly, tracking the "Recovery-Phase" relies on the AI mapping sentiment volatility, detecting extreme spikes of verbal aggression followed immediately by intense, disproportionate affection designed to stabilize the victim.

Domain 2: Evasion, Dismissal, and Emotional Unavailability

#	Metric	Clinical Definition & AI Detection Mechanism
11	Stonewalling	Complete refusal to communicate during a conflict. The AI flags instances where lengthy emotional paragraphs are met with singular, dismissive responses or absolute silence.
12	Ghosting /Breadcrumbing	Sporadic, non-committal communication. The AI calculates extreme gaps in message timestamps followed by low-effort engagement (e.g., sending a single emoji after days of silence).
13	Dismissiveness	Invalidating the partner's

#	Metric	Clinical Definition & AI Detection Mechanism
		emotional state. The AI utilizes sentiment classification to detect phrases like "You're overreacting" or "You're acting like a child".
14	Deflection	Avoiding accountability by changing the subject. The LLM monitors semantic drift, identifying when a direct question is continually sidestepped.
15	Weaponized Incompetence	Feigning inability to perform basic tasks. The AI flags recurring text patterns where the initiator claims confusion or inability to force the partner to assume the physical or emotional burden.
16	Chronic Sarcasm	Utilizing humor to deliver insults. The deep learning model parses the context to differentiate between playful banter and hostile, passive-aggressive sarcasm used to maintain plausible deniability.
17	Intellectualization	Responding to emotional pleas with cold, hyper-logical counter-arguments. The AI measures the total absence of emotional lexicon in response to high-distress messages.
18	Minimization	Aggressively downplaying the impact of an event. The AI flags statements acknowledging an event occurred but denying the psychological harm inflicted on the partner.
19	Tone Policing	Ignoring the substance of a message to critique how it was expressed. The AI identifies when the response focuses entirely on the sender's perceived anger rather than the topic at hand.
20	Intentional Ambiguity	Deliberately withholding clear answers to maintain a power imbalance. The AI evaluates the ratio of vague, non-

#	Metric	Clinical Definition & AI Detection Mechanism
		committal statements to direct answers.

Evasive metrics are deeply tied to the psychological concept of chronemic expectancy violations. Stonewalling and ghosting are not merely about the words said, but the weaponization of time. The LLM will analyze the metadata timestamps to determine if a partner deliberately induces anxiety by delaying responses during critical conflict scenarios, a tactic that heavily correlates with relationship deterioration and emotional burnout.

Domain 3: Linguistic, Structural, and Chronemic Data Markers

#	Metric	Clinical Definition & AI Detection Mechanism
21	Response Time Latency	Analyzing the mathematical variance in response times. The AI flags asymmetries, such as a partner who takes 12 hours to reply but demands instant answers from the user.
22	Word Count Asymmetry	Calculating the ratio of words sent versus words received. A chronic 80/20 split indicates severe conversational dominance or profound emotional withdrawal.
23	Initiation Ratio	Tracking which partner initiates conversations most frequently. The AI flags highly unbalanced relational effort, identifying one-sided engagement.
24	Repair-to-Rupture Ratio	Measuring the time elapsed between a high-conflict spike and the first de-escalation attempt. Fast repairs correlate with high relationship survival; delayed repairs signal toxicity.
25	Question-to-Statement Ratio	Evaluating reciprocity. The AI flags a lack of incoming questions, indicating a profound lack of curiosity or empathy regarding the user's daily life or emotional state.
26	Pronoun Dominance	Excessive use of "I/me/my" versus "we/us". The AI calculates this ratio as a strong linguistic indicator of underlying narcissistic personality behaviors.
27	Punctuation Aggression	Algorithmic tracking of digital

#	Metric	Clinical Definition & AI Detection Mechanism
		shouting (ALL CAPS) or passive-aggressive punctuation, such as the abrupt use of periods in normally casual syntax environments.
28	Message Clustering	Sending dozens of messages in rapid succession to overwhelm the recipient before they can process or respond to the information.
29	Time-of-Day Volatility	Identifying patterns of conflict instigation occurring exclusively during late night or high-stress hours, utilizing the message timestamp metadata.
30	Mismatched Bids for Connection	The AI tracks when one partner reaches out emotionally and the other completely misses or ignores the bid, a primary indicator of relationship deterioration.

This domain relies less on semantic interpretation and heavily on quantitative data processing. The AI functions as a statistical engine, parsing the mathematical structure of the relationship. For example, analyzing the "Repair-to-Rupture Ratio" requires the LLM to identify the exact timestamp a hostile argument ceases and calculate the duration until a repair attempt (an apology or de-escalation) is made. Couples who attempt repair within four hours demonstrate high survival rates, whereas leaving ruptures unaddressed overnight is a critical metric of dysfunction.

Domain 4: Overt Aggression and Devaluation

#	Metric	Clinical Definition & AI Detection Mechanism
31	Name Calling	Explicit insults and derogatory labeling. The AI utilizes toxicity classifiers to flag direct verbal abuse (e.g., "You're such an idiot").
32	Threats of Abandonment	Utilizing the relationship status as a weapon. The AI flags phrases like "I can't do this anymore" or "I'm breaking up with you" used repeatedly as leverage.
33	Character Assassination	Attacking the partner's core identity rather than addressing a specific behavior. The AI differentiates between

#	Metric	Clinical Definition & AI Detection Mechanism
		behavioral critique and identity attacks.
34	Contempt	Expressing absolute disgust or superiority over the partner. The AI detects mockery and superiority complexes, the highest predictor of divorce and relationship failure.
35	Public Humiliation Threats	Sending messages that reference or threaten public embarrassment or the sharing of private information to force the partner into submission.
36	Coercion	Pressuring the partner into actions they have explicitly declined. The AI flags repeated requests for a boundary to be crossed after a clear "no" has been established.
37	Victim Blaming	Holding the partner responsible for the initiator's abusive actions. The AI detects phrases shifting fault, such as "Look what you made me do."
38	Baiting	Intentionally sending provocative messages designed to elicit an angry response, often to later accuse the partner of being "crazy" or unstable.
39	Pathologizing	Diagnosing the partner with mental illnesses as a form of control. The AI flags weaponized clinical terms (e.g., "You're bipolar," "You need medication").
40	Retaliatory Anger	Responding to a mild critique with vastly disproportionate rage. The AI maps the emotional escalation curve between the prompt and the response.

Overt aggression is the most easily detectable domain for traditional NLP tools, but modern LLMs like DeepSeek V3.2 provide critical contextual oversight. The AI must determine if the aggression is unilateral or reactive. A victim utilizing harsh language in self-defense after prolonged abuse must not be miscategorized as the primary aggressor. The model achieves this by mapping the entire conversation history, identifying the instigator of the initial hostile intent and recognizing baiting tactics designed specifically to provoke a retaliatory response.

Domain 5: Boundary Violations and Intrusiveness

#	Metric	Clinical Definition & AI Detection Mechanism
41	Surveillance / Tracking	Demanding proof of location or immediate status updates. The AI flags imperative commands regarding physical location (e.g., "Send me a photo so I know where you are").
42	Floodlighting / Trauma Dumping	Releasing unprocessed trauma onto the partner without warning early in the relationship to force a false sense of intimacy and overwhelm boundaries.
43	Digital Intrusiveness	Utilizing text to demand passwords, access to the partner's phone, or questioning their secondary communications with friends and family.
44	Sleep Deprivation Tactics	Deliberately initiating high-stakes arguments via text late at night and refusing to let the partner sleep until the instigator's demands are resolved.
45	Interference with Work/Social Life	Texting incessantly during known working hours or social engagements. The AI correlates high message volume with traditional working hours to detect sabotage.
46	Overstepping Autonomy	Dictating what the partner is allowed to wear, who they can see, or how they spend their money via text-based commands and prohibitions.
47	Forced Forgiveness	Demanding immediate absolution after an offense. The AI flags situations where the abuser refuses to allow the partner time to process the emotional event.
48	Double Standards	Establishing rules for the partner's text etiquette (e.g., fast response times) that the initiator blatantly ignores in their own communication patterns.

#	Metric	Clinical Definition & AI Detection Mechanism
49	Pacing Violations	Attempting to force the relationship to progress (moving in, marriage) at an unnatural, pressurized speed via constant messaging and future-faking.
50	Accountability Deflection	Emitting apologies that are fundamentally void of responsibility. The AI flags "non-apologies" such as "I'm sorry you feel that way," which place the blame on the victim's reaction rather than the action.

This final domain captures the essence of systemic control. Identifying concepts like Floodlighting or Trauma Dumping requires the LLM to evaluate the depth and appropriateness of the shared content relative to the chronological age of the relationship. Sharing deep trauma on day two of a texting timeline is a boundary violation; sharing it on day three hundred is healthy intimacy. The AI dynamically weighs the severity of these metrics against the overall timestamp architecture of the thread, ensuring the analysis reflects clinical realities rather than rigid keyword matching.

Core Application Logic and Pseudocode Specifications

The complexity of orchestrating native device ingestion, Unipile REST API callbacks, DeepSeek LLM inference, and highly formatted PDF rendering requires an immaculate state management architecture. The application utilizes a declarative UI model in Flutter, segregating the business logic from the visual rendering pipeline.

The left pane operates as the user control center. It manages the OAuth connections to third-party providers via Unipile, displays the available email and chat threads, and renders a toggleable checklist allowing the user to select exactly 10 of the 50 psychological metrics for the current analysis session. The right pane functions as the real-time document viewer, displaying the generative stream from the AI and subsequently rendering the interactive PDF utilizing the pdf package's pw.Document structure.

The following exhaustive pseudocode delineates the critical pathways governing data ingestion, API construction, and document synthesis.

```
//
=====
===
// CORE DATA STRUCTURES & MODELS
//
=====
===

class ChatMessage {
  final String messageId;
  final String senderIdentifier;
```

```

final String textBody;
final DateTime timestamp;
final List<Uint8List> mediaAttachments; // Stores binary image data for PDF
final bool isInitiator;

  ChatMessage({this.messageId, this.senderIdentifier, this.textBody, this.timestamp,
this.mediaAttachments, this.isInitiator});
}

class ConversationThread {
  final String threadId;
  final String platformSource; // "SMS", "WhatsApp", "Gmail", "LinkedIn"
  final List<ChatMessage> messages;

  ConversationThread({this.threadId, this.platformSource, this.messages});
}

class PsychologicalAnalysisReport {
  final String executiveSummary;
  final Map<String, double> metricScores; // Evaluated metrics scaled 0.0 to 1.0
  final List<String> evidentiaryQuotes;
  final String forensicVerdict;

  PsychologicalAnalysisReport({this.executiveSummary, this.metricScores,
this.evidentiaryQuotes, this.forensicVerdict});
}

//
=====
====
// DATA INGESTION SERVICES (TIER 1 & TIER 2)
//
=====
====

class NativeExtractionService {
  // Parses iOS SQLite backups or Android content resolver payloads for Tier 1 users
  Future<ConversationThread> parseDeviceSQLiteDatabase(File databaseFile) async {
    // Connects to local SQLite instance extracted via desktop software
[span_160](start_span)[span_160](end_span)[span_161](start_span)[span_161](end_span)
    var rawQueryData = await SQLiteInterface.rawQuery(
      databaseFile,
      "SELECT text, date, is_from_me, handle_id FROM message WHERE text IS NOT NULL"
    );
    return mapQueryToConversationThread(rawQueryData, platform: "Native SMS");
  }
}

class UnipileIntegrationService {
  // Manages Tier 2 premium access to encrypted networks and email
[span_162](start_span)[span_162](end_span)[span_163](start_span)[span_163](end_span)

```

```

final String unipileApiKey = "UNIPILE_SECURE_KEY_LADY_LAKE";

Future<List<ConversationThread>> fetchPremiumThreads(String accountId) async {
    // GET request interfaces with Unipile's unified schema for WhatsApp, Gmail, etc.
[span_164](start_span)[span_164](end_span)
    var httpResponse = await http.get(
        url: Uri.parse("https://api.unipile.com/api/v1/chats?account_id=$accountId"),
        headers: {"Authorization": "Bearer $unipileApiKey"}
    );

    if (httpResponse.statusCode == 200) {
        return parseUnipileJsonPayload(httpResponse.body);
    } else {
        throw Exception("Unipile Sync Failure: Verify API node status.");
    }
}

//
=====
// ARTIFICIAL INTELLIGENCE INFERENCE ENGINE
//
=====

class DeepSeekInferenceEngine {
    final String llmApiEndpoint = "https://api.deepseek.com/v1/chat/completions";
    final String deepSeekKey = "DEEPSEEK_API_KEY";

    Future<PsychologicalAnalysisReport> executeForensicAnalysis(ConversationThread
targetThread, List<String> selectedMetrics) async {
        // Validates that exactly 10 metrics are passed to the inference engine
        assert(selectedMetrics.length == 10, "Strictly 10 metrics must be selected for analysis.");

        String structuredChatTranscript = serializeChatForPrompt(targetThread);

        // System prompt engineered for strict psychological evaluation
[span_165](start_span)[span_165](end_span)[span_166](start_span)[span_166](end_span)
        String rigorousSystemPrompt = """
        You are an expert digital forensic psychologist. You are tasked with analyzing the following
communication timeline.
        You must strictly evaluate the interaction against the following 10 clinical metrics:
        ${selectedMetrics.join(", ")}.
        Do not invent metrics. Do not provide polite affirmations. Be analytical, brutal, and precise.
        Return the analysis strictly as a JSON object containing:
        1. "executiveSummary": A brief clinical overview.
        2. "metricScores": A map of the 10 metrics with a float score from 0.0 (benign) to 1.0 (highly
toxic).
        3. "evidentiaryQuotes": An array of specific strings extracted from the text proving the
toxicity.

```

4. "forensicVerdict": The final relationship diagnosis.

```
""",
    },

    var jsonPayload = {
        "model": "deepseek-v3.2", // Selected for massive context window and cost-efficiency
[span_167](start_span)[span_167](end_span)[span_168](start_span)[span_168](end_span)
        "messages":,
        "temperature": 0.15, // Ultra-low temperature enforces deterministic, JSON-compliant
outputs
        "response_format": {"type": "json_object"}
    };

    var apiResponse = await http.post(
        Uri.parse(IIlmaApiEndpoint),
        headers: {"Content-Type": "application/json", "Authorization": "Bearer $deepSeekKey"},
        body: jsonEncode(jsonPayload)
    );

    return deserializeDeepSeekResponseToModel(apiResponse.body);
}
}

//
=====
====
// PDF SYNTHESIS AND RENDER PIPELINE
//
=====
=====

class PdfSynthesisEngine {
    // Utilizes Flutter 'pdf' and 'printing' packages to draw directly to a binary stream
[span_169](start_span)[span_169](end_span)[span_170](start_span)[span_170](end_span)
    Future<File> constructForensicDocument(ConversationThread thread,
PsychologicalAnalysisReport report) async {
        final pdfDocument = pw.Document();

        pdfDocument.addPage(
            pw.MultiPage(
                pageFormat: PdfPageFormat.a4,
                margin: pw.EdgeInsets.all(32),
                build: (pw.Context context) =>
                ),
            pw.SizedBox(height: 20),

            // Extracted Evidentiary Quotations
            pw.Header(level: 1, child: pw.Text("Flagged Evidentiary Quotations")),
            ...report.evidentiaryQuotes.map((quote) => pw.Bullet(text: quote, style:
pw.TextStyle(fontStyle: pw.FontStyle.italic))).toList(),
            pw.Divider(),
        );
    }
}
```



```

        // Full Transcript with Embedded Binary Images
[span_171](start_span)[span_171](end_span)
        pw.Header(level: 1, child: pw.Text("Complete Communication Transcript")),
        ...thread.messages.map((msg) {
            return pw.Container(
                margin: pw.EdgeInsets.only(bottom: 5),
                alignment: msg.isInitiator? pw.Alignment.centerRight : pw.Alignment.centerLeft,
                child: pw.Column(
                    crossAxisAlignment: msg.isInitiator? pw.CrossAxisAlignment.end :
pw.CrossAxisAlignment.start,
                    children: [
                        pw.Text(msg.textBody),
                        // Render inline binary image attachments if present
                        if (msg.mediaAttachments.isNotEmpty)
                            pw.Image(pw.MemoryImage(msg.mediaAttachments.first), width: 150),
                    ],
                ),
            );
        }).toList(),
    ],
),
);

return await saveBinaryToLocalStorage(pdfDocument.save());
}
}

//
=====
====
// MASTER UI TOPOGRAPHY: SPLIT PANE ARCHITECTURE
//
=====
=====

class ToxicityAnalyzerMasterView extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, boxConstraints) {
        // Enforces Left/Right pane UI on tablets and web/desktop deployments
        if (boxConstraints.maxWidth > 850) {
          return Row(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children:
          );
        } else {
          // Graceful degradation to a stacked navigation stack for mobile phones
          return MobileNavigationStack();
        }
      }
    );
  }
}

```

```

    );
  }
}

class DashboardControlPane extends StatefulWidget {
  // Houses OAuth buttons, thread lists, and the 50-metric checklist
  void executeFullAnalysisPipeline() async {
    // 1. Subscription Verification Gate
    if (!SubscriptionManager.hasActiveSubscription) {

triggerAppStorePurchase[span_10](start_span)[span_10](end_span)Overlay(subscriptionTier:
"\$9.99/mo Base Tier");
      return;
    }

    // 2. State configuration
    List<String> chosenMetrics = MetricState.getSelected(limit: 10);
    ConversationThread activeThread = AppState.activeSelection;
    AppState.isProcessing = true;
    updateUI();

    // 3. Inference and Synthesis Execution
    PsychologicalAnalysisReport generatedReport = await
DeepSeekInferenceEngine().executeForensicAnalysis(activeThread, chosenMetrics);
    File finalPdfBinary = await PdfSynthesisEngine().constructForensicDocument(activeThread,
generatedReport);

    // 4. State resolution and Right Pane rendering
    AppState.activeReport = generatedReport;
    AppState.activePdfDocument = finalPdfBinary;
    AppState.isProcessing = false;
    updateUI();
  }
}

```

The logic strictly enforces the selection of exactly ten metrics, preventing prompt overload and ensuring the DeepSeek model applies maximum attention density to the chosen behavioral traits. By isolating the PDF generation into a dedicated service that natively processes binary memory arrays (Uint8List), the application successfully bypasses the limitations of traditional web-wrapper PDF generators, allowing seamless inclusion of MMS and WhatsApp image attachments.

Corporate Formation, Compliance, and Financial Infrastructure

Operating a global SaaS application with dual-tier recurring revenue (\$9.99 and \$19.99 per month) against a continuously fluctuating cost of goods sold (DeepSeek API tokens and Unipile API nodes) necessitates a highly structured corporate and banking foundation. The mandate dictates locating this operational nexus in Lady Lake, Florida. Lady Lake is a municipality uniquely straddling the borders of Lake County and Sumter County, requiring specific

jurisdictional compliance to legally route App Store revenue into commercial bank accounts.

Corporate Entity Formation and Municipal Registration

The absolute first priority is establishing a corporate veil, ideally a Limited Liability Company (LLC), to shield the application's founder from algorithmic liability or user data disputes.

1. **State-Level Registration:** The LLC must be formally registered with the Florida Department of State via the Sunbiz.org portal. The founder will file the "Articles of Organization," designating a physical address in Lady Lake and a registered agent in Florida.
2. **Federal Taxation Footprint:** Once the Florida Division of Corporations registers the entity, the business must immediately apply for an Employer Identification Number (EIN) through the Internal Revenue Service (IRS). The EIN serves as the federal anchor required by all commercial banks and App Store developer programs.
3. **Local Municipal Compliance:** Because the physical operations are based in Lady Lake, the LLC is subject to local zoning and taxation laws. All businesses operating within Sumter County must obtain a Local Business Tax Receipt from the Sumter County Tax Collector's Office (or the Lake County equivalent, dependent on the exact street address). The application process involves a \$50 non-refundable clearance fee and requires zoning approval. If the software development occurs in a residential home in Lady Lake, a notarized property owner clearance form is mandatory.

Strategic Banking Selection

Lady Lake and the surrounding central Florida region offer a plethora of financial institutions capable of anchoring the application's capital flow. The primary concern for this application is identifying a bank that supports high-frequency, automated API disbursements without incurring prohibitive transactional fees.

Wells Fargo remains the highest-rated national bank for small businesses operating in Florida, offering robust commercial checking accounts, direct integrations with accounting platforms, and immense institutional stability. Truist Bank offers a tiered "Dynamic Business" account that waives its \$20 monthly maintenance fee if the developer maintains a \$5,000 average combined balance, which is easily achievable once the App Store payouts stabilize. However, for a digital-native startup, blending a local account at a regional institution like MIDFLORIDA Credit Union (which offers a highly accessible "Freedom Business Checking" tier for new startups) with an advanced digital fintech platform like BlueVine or Novo often yields the best results. Digital platforms like BlueVine offer integrated virtual debit cards, which allow the developer to isolate AI API payments (DeepSeek) from backend server payments (Unipile) for highly granular expense tracking.

App Store Revenue Recognition and Capital Routing

With the corporate entity formalized and the Lady Lake bank account active, the application must be monetized across the Apple and Google ecosystems.

Within the Apple ecosystem, the LLC utilizes its newly acquired EIN and D-U-N-S Number to enroll in the Apple Developer Program as a corporate organization rather than an individual. The developer signs "Schedule 2" (the Paid Apps Agreement) in App Store Connect, mapping the incoming subscription revenue directly to the routing and account numbers of the Lady Lake bank account. To maximize the operational margin available for purchasing API tokens, the LLC must aggressively apply for the Apple Small Business Program, which cuts Apple's standard

30% commission down to 15% for the first \$1 million in annual revenue. Google Play offers a similar 15% tier for the first million dollars through its Play Console payments profile. The final architectural hurdle involves automated accounting. Apple and Google do not deposit money the moment a user subscribes. They aggregate cash payments and distribute them globally on a delayed monthly schedule, net of platform fees and international taxes. Because the application consumes Unipile API nodes and DeepSeek tokens daily, cash flow must be tracked continuously. Implementing a billing and revenue recognition middleware—such as Chargebee or RevRec—automates compliance with the ASC 606 revenue recognition standard. These platforms sync the opaque, aggregated App Store Subscriber Reports into highly detailed, transaction-level data, allowing the Lady Lake operation to accurately forecast its net cash position and ensure uninterrupted funding for the underlying AI infrastructure.

Addendum: Tier 3 Forensics - Location and Travel History Analysis (\$50 Consumable IAP)

To further expand the evidentiary capabilities of the application, a third analytical pillar allows users to scrutinize the travel patterns of a consenting partner. This tier is not tied to the monthly subscription structure; instead, it is gated behind a \$50 one-time, consumable In-App Purchase (IAP). Using behavioral AI models, the application analyzes historical geolocation data to identify chronemic anomalies—such as travel at odd hours, unexplained gaps, or highly frequent, regular trips to unexplained residential locations.

Financial Engineering for the \$50 IAP

Because this is a per-analysis fee, the Flutter application must integrate a "Consumable" product architecture rather than an auto-renewing subscription. Utilizing the `in_app_purchase` Flutter package, or a managed middleware solution like RevenueCat, the app will query the Apple App Store and Google Play Store for the specific \$50 product ID. The analysis button on the frontend remains visually disabled (greyed out) until the transaction receipt is validated by the OS. Once the user completes the transaction, the UI state updates, unlocking the biometric or password prompt to begin the ingestion phase.

Technical Limitations and the "Sign-In" Workaround (2026 Privacy Compliance)

The user's initial proposal—having the consenting partner simply type their Google or Apple ID credentials into the app to instantly fetch location history—is no longer technically feasible due to severe architectural and legal shifts implemented by both Apple and Google in late 2024 through 2026.

1. **The Google Timeline Shift:** In December 2024, Google fundamentally altered how it handles Google Maps "Timeline" (Location History) data. To settle ongoing privacy lawsuits, Google deleted the web-based Timeline portal entirely and moved all historical location data to be stored strictly *locally* on the user's mobile device. Because the data is now decentralized and device-bound, an OAuth "Sign in with Google" API call will no longer return a user's full, multi-year location history.
2. **Apple Significant Locations:** Similarly, Apple's iOS tracking feature, "Significant Locations," is end-to-end encrypted. Apple explicitly prohibits third-party applications from querying this historical tracking database via an API to prevent covert surveillance.

The Implementation Pathway: To bypass these strict API limitations while maintaining explicit

user consent, the application will provide a guided, in-app tutorial prompting the consenting partner to manually provide the raw data logs to the device running the analysis.

- **For Android/Google Users:** The app will guide the consenting partner to use "Google Takeout" on their device to export their Location History. The resulting JSON file (Semantic Location History) will then be imported directly into the Flutter app.
- **For Apple/iOS Users:** The app will prompt the user to utilize the same desktop-extraction software (like iMazing or iBackup Extractor) used for the Tier 1 SMS extraction. This desktop interface can pull the encrypted location logs from a local iTunes/Finder backup, generating a CSV that can be fed into the toxicity analyzer.

AI Behavioral Analysis Methodology

Once the raw JSON or CSV geographic data is imported, the application will not simply plot points on a map. Instead, it utilizes the DeepSeek V3.2 LLM as a Behavioral Threat Detection engine, similar to advanced cybersecurity tools.

The system prompt will instruct the AI to execute the following pipeline:

1. **Baseline Establishment:** The AI will mathematically determine the user's "normal" geographic behavior (e.g., standard commuting routes between the primary residence and workplace, typical hours of transit).
2. **Anomaly Detection:** Once the baseline is established, the AI will scan the timeline for significant deviations. It will specifically search for chronemic expectancy violations (e.g., a device registering movement at 3:00 AM on a Tuesday when the baseline dictates the user should be asleep).
3. **Pattern Recognition:** The AI will isolate geographic coordinates that do not match known commercial addresses (using reverse-geocoding APIs) to identify frequent, recurring visits to secondary residential locations on specific days of the week.

The final output is appended to the master PDF report, citing specific dates, times, and coordinate variances that violate the established behavioral baseline, providing the user with a highly structured map of geographic irregularities.

Works cited

1. Android Example B4Xgoodies_from_walt61 Online, https://www.b4x.com/android/forum/threads/b4xgoodies_from_walt61-online.110060/page-11
2. React Native vs Flutter vs Native for Healthcare Apps: What to Choose in 2026, <https://seesawlabs.com/the-lab/guides/react-native-vs-flutter-vs-native-healthcare-2026>
3. Flutter vs React Native: Which Is Better for Cross-Platform App Development in 2026?, <https://dev.to/prateekshaweb/flutter-vs-react-native-which-is-better-for-cross-platform-app-development-in-2026-1e5f>
4. Flutter in 2026: Building High-Fidelity Apps in an AI-Augmented World - Medium, <https://medium.com/@shreebhagwat94/how-would-i-learn-flutter-in-2026-the-future-of-app-development-f7a362f5acb9>
5. Flutter PDF Library | PDF Generator API - Syncfusion, <https://www.syncfusion.com/document-sdk/flutter-pdf-library>
6. pdf | Dart package - Pub.dev, <https://pub.dev/packages/pdf>
7. Generating Dynamic PDF Reports in Flutter with printing and pdf Packages | by Navin Prince, <https://medium.com/@navinprince.thegr8gossiper/generating-dynamic-pdf-reports-in-flutter-with-printing-and-pdf-packages-acd251ea2616>
8. Flutter Tutorial - Create PDF With Images & Tables - YouTube, https://www.youtube.com/watch?v=C5c_AuhbzOo
9. Generating PDF file with Images in flutter - Stack Overflow, <https://stackoverflow.com/questions/67718947/generating-pdf-file-with-images-in-flutter>
10. Use of SMS or Call Log permission groups - Play Console Help,

<https://support.google.com/googleplay/android-developer/answer/10208820?hl=en> 11. Policy announcement: April 10, 2025 - Play Console Help - Google Help, <https://support.google.com/googleplay/android-developer/answer/15899442?hl=en> 12. Google To Manually Approve Permissions For SMS & Phone Apps, <https://appetiser.com.au/blog/google-to-manually-approve-permissions-for-sms-and-phone-apps/> 13. Update on apps distributed in the European Union - Support - Apple Developer, <https://developer.apple.com/support/dma-and-apps-in-the-eu/> 14. Complying with the Digital Markets Act | Apple Developer, <https://developer.apple.com/security/complying-with-the-dma.pdf> 15. Exploring, in Detail, Apple's Compliance With the EU's DMA Mandate Regarding Apple Watch, Third-Party Accessories, and the Syncing of Saved Wi-Fi Networks From iPhones to Which They're Paired - Daring Fireball, https://daringfireball.net/2025/11/apple_eu_dma_iphone_accessories_wi-fi_sync 16. iOS 19 and iOS 20 Must Include a Long List of Major Changes, EU Says : r/apple - Reddit, https://www.reddit.com/r/apple/comments/1jf3ruk/ios_19_and_ios_20_must_include_a_long_list_of/ 17. Interoperability - Digital Markets Act (DMA) - European Commission, https://digital-markets-act.ec.europa.eu/developer-portal/interoperability_en 18. How To Export iPhone Messages (SMS, RCS, iMessage) to your computer in PDF, Excel, CSV, and RSMF formats - iMazing, <https://imazing.com/guides/how-to-export-iphone-text-messages-sms-and-imessages-to-your-computer-as-pdf-excel-csv-or-rsmf> 19. Extract Messages from iPhone Backup - Wide Angle Software, <https://www.wideanglesoftware.com/ibackupextractor/help/extract-messages-from-iphone-backup.php> 20. Best Tool For Extracting iMessages and SMS messages from iPhone : r/ios - Reddit, https://www.reddit.com/r/ios/comments/1fumvmw/best_tool_for_extracting_imessages_and_sms/ 21. How to Transfer Text Messages from iPhone to Computer (2 Ways Including Free Solutions), <https://www.youtube.com/watch?v=fTyZAA5Sqyo> 22. Unified Inbox and Cross-Platform Communication - Unipile, <https://www.unipile.com/unified-inbox-and-communication/> 23. Best Nylas Alternative for Email and Calendar API in 2026 - Unipile, <https://www.unipile.com/alternative-nylas/> 24. Unipile: One API for Multi-Channel Messaging in your app, <https://www.unipile.com/> 25. Messaging API with Unipile in Minutes, <https://www.unipile.com/communication-api/messaging-api/> 26. Boost Outreach Efficiency with Messaging and Email Automation - Unipile, <https://www.unipile.com/messaging-and-email-automation/> 27. Send Email API — Gmail, Outlook & IMAP from User Accounts (2026) - Unipile, <https://www.unipile.com/send-email-api/> 28. Retrieve WhatsApp Chat History (GetChatHistory) - green-api, <https://green-api.com/en/docs/api/journals/GetChatHistory/> 29. First Bank of the Lake, <https://www.fblake.bank/> 30. Email API to Sync, Send & Receive Emails in Your App - Unipile, <https://www.unipile.com/communication-api/email-api/> 31. IMAP API Integration for Your App - Unipile, <https://www.unipile.com/communication-api/email-api/imap-api/> 32. Multi-Provider Email API: Connect Gmail, Outlook and IMAP with One Integration - Unipile, <https://www.unipile.com/unified-email-api-integration/> 33. Email API for Microsoft Outlook Integration - Unipile, <https://www.unipile.com/email-api-for-microsoft-outlook-integration/> 34. WhatsApp API Pricing: Understand Costs and Find a Scalable Alternative - Unipile, <https://www.unipile.com/whatsapp-api-pricing-understand-costs-and-find-a-scalable-alternative/> 35. LLM API Pricing 2026 — Compare GPT-5, Claude 4, Gemini 2.5, DeepSeek Costs | TLDL, <https://www.tldl.io/resources/llm-api-pricing-2026> 36. Low Cost LLM API: 2026 Price & Performance Guide, <https://aisuperior.com/low-cost-llm-api/> 37. LLM API Pricing Comparison (2025): OpenAI, Gemini, Claude | IntuitionLabs, <https://intuitionlabs.ai/articles/llm-api-pricing-comparison-2025> 38. LLM API Pricing Comparison 2026: Every Major Model, Ranked by Cost | BenchLM.ai, <https://benchlm.ai/blog/posts/llm-pricing-2026> 39. Cheapest LLM API (April 2026) — DeepSeek \$0.14 vs Gemini Flash-Lite \$0.10 Per 1M Tokens | TLDL, <https://www.tldl.io/resources/cheapest-llm-api-2026> 40. AI vs. Traditional Methods: Detecting Toxic Communication - Gaslighting Check, <https://www.gaslightingcheck.com/blog/ai-vs->

traditional-methods-detecting-toxic-communication 41. Scientists develop AI tool to weed out gaslighters and narcissists - Cybernews, <https://cybernews.com/ai-news/ai-tool-spots-gaslighters-narcissists-abuse/> 42. LLM API Pricing Comparison & Cost Guide (Apr 2026) - CostGoat, <https://costgoat.com/compare/llm-api> 43. Enhancing LLM-based Hatred and Toxicity Detection with Meta-Toxic Knowledge Graph - arXiv, <https://arxiv.org/html/2412.15268v4> 44. Toxic Relationship Text Patterns: Red Flags in Your Messages | MosaicChats, <https://www.mosaicchats.com/blog/toxic-relationship-text-patterns> 45. Detecting Hidden Harm: Using AI to Spot Emotional Abuse in Text Messages, <https://huggingface.co/blog/SamanthaStorm/tetherai> 46. What 36,000 Couples Taught an AI About Love - Empathi.com, <https://empathi.com/blog/ai-relationship-data-insights/> 47. (PDF) No response? Chronemic expectancy violation and relational turbulence in technologically-mediated romantic relational conflict - ResearchGate, https://www.researchgate.net/publication/375589577_No_response_Chronemic_expectancy_violation_and_relational_turbulence_in_technologically-mediated_romantic_relational_conflict 48. Common Modern Dating Red Flags: What They Really Mean and How to Spot Them, <https://www.summitcs.ca/edmontontherapyblog/common-modern-dating-red-flags-what-they-really-mean-and-how-to-spot-them> 49. Signs Your Partner Is Toxic Texting You - Jan and Jillian, <https://www.janandjillian.com/relationship/signs-your-partner-is-toxic-texting-you> 50. 10 Toxic Dating Terms You Need to Know (and What They Really Mean), <https://fatherhoodchannel.com/2025/05/10/10-toxic-dating-terms-you-need-to-know-and-what-they-really-mean/> 51. How to Build a Pseudocode Runner with AI - DEV Community, <https://dev.to/devin-rosario/how-to-build-a-pseudocode-runner-with-ai-11nk> 52. Starting a Business in Sumter County Florida, <https://www.floridasmart.com/starting-a-business-in-sumter-county-florida> 53. Business License | City of Sumter, SC, <https://www.sumtersc.gov/business-license> 54. Start a Business - Division of Corporations - Florida Department of State, <https://dos.fl.gov/sunbiz/start-business/> 55. Standard registration process for non-regulated businesses - Open MyFlorida Business, <https://openmyfloridabusiness.gov/business/99/standard-registration-process-for-non-regulated-businesses/> 56. Doing Business in the State of Florida, <https://www.stateofflorida.com/corporations/> 57. How to Register your business in Florida - (Sunbiz.org) - YouTube, <https://www.youtube.com/watch?v=Q6hgDR8sVWo> 58. Small Business Bank Accounts, Loans & Credit Cards | Truist, <https://www.truist.com/small-business> 59. Valley Bank, <https://www.valley.com/> 60. Hancock Whitney Bank - Checking, Savings, Loans - Personal & Business, <https://www.hancockwhitney.com/> 61. 2026's Best Banks for Small Business | SmartAsset.com, <https://smartasset.com/checking-account/best-banks-for-small-business> 62. Best Banks In Florida For 2026 | Bankrate, <https://www.bankrate.com/banking/best-banks-in-florida/> 63. Free Business Checking Account - MIDFLORIDA Credit Union, <https://www.midflorida.com/business/checking/free-business-checking> 64. Novo vs Wells Fargo vs Truist — Best Bank For Small Business Checking Accounts, <https://jamesgreenebc.medium.com/novo-vs-wells-fargo-vs-truist-best-bank-for-small-business-checking-accounts-80a16386f099> 65. What banks do you guys recommend for small businesses just starting? - Reddit, https://www.reddit.com/r/smallbusiness/comments/1ip1s4z/what_banks_do_you_guys_recommend_for_small/ 66. Tranfering Apps between accounts (Apple and Google) - RevenueCat Community, <https://community.revenuecat.com/general-questions-7/tranfering-apps-between-accounts-apple-and-google-7448> 67. App Store Small Business Program - Apple Developer, <https://developer.apple.com/app-store/small-business-program/> 68. Enter banking information - Manage banking information - App Store Connect - Help - Apple Developer, <https://developer.apple.com/help/app-store-connect/manage-banking-information/enter-banking-information/> 69. Integrating Apple App Store - RevRec - Chargebee, <https://www.chargebee.com/docs/revrec/integrations/apple> 70. App Store Commissions: An

ASC 606 Accounting Guide - HubiFi, <https://www.hubifi.com/blog/accounting-for-revenue-recognition-for-asc-606>